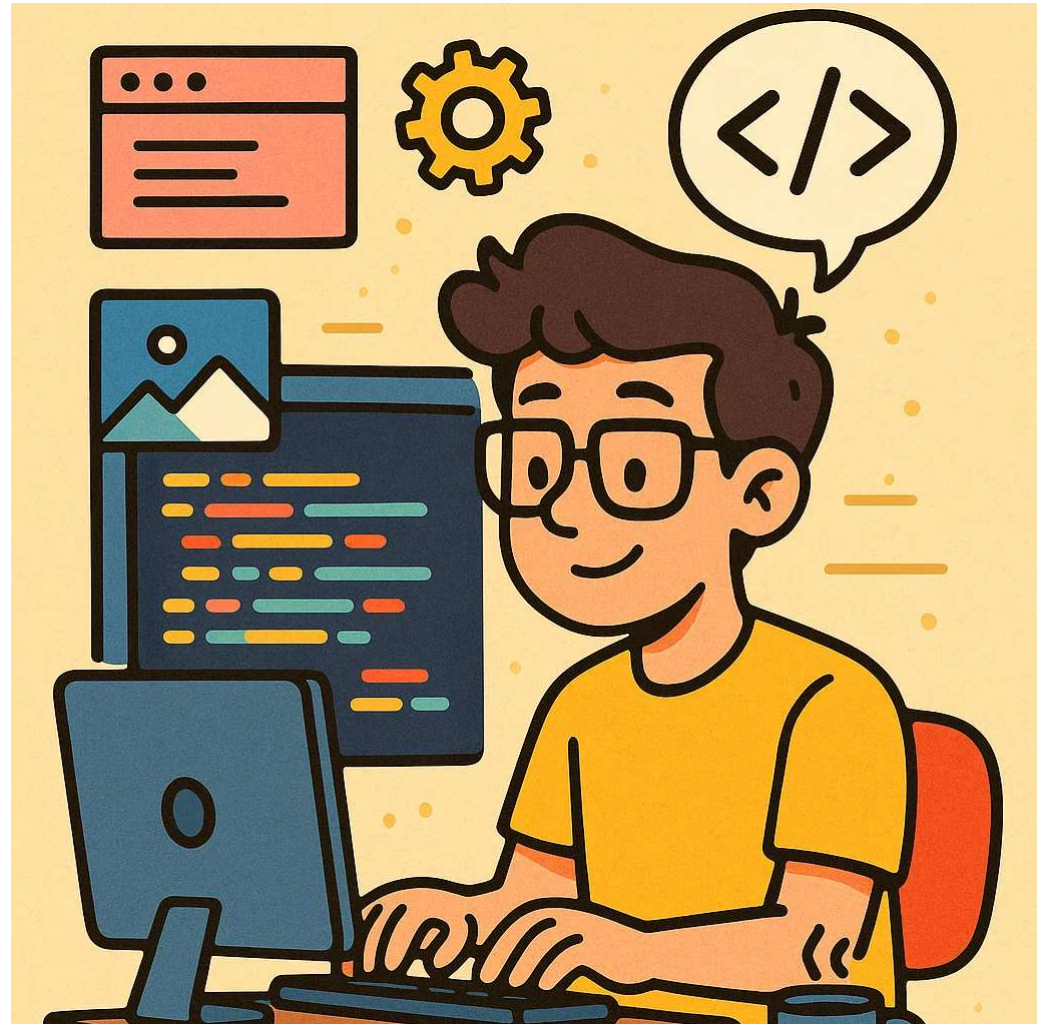


CRUD em C# Console com SQL Server

Professor Danilo



Introdução

- Nesta aula, vamos criar um sistema simples de gerenciamento de alunos usando C#, SQL Server e POO.
- Serão implementadas todas as operações CRUD: Create, Read, Update e Delete.
- CRUD é a base de qualquer sistema que manipula dados. O foco será conectar o console C# a um banco de dados SQL Server e organizar o código com POO.



Objetivos

- Compreender a integração de C# com SQL Server, aplicar conceitos de POO e executar operações básicas de CRUD.
- Ao final, você saberá criar, listar, atualizar e excluir registros de um banco de dados, mantendo o código organizado e reutilizável.

Estrutura do Projeto

Dividiremos o projeto em camadas para organizar o código:

- Models → Classes que representam os dados;
- Repository → Classe que acessa o banco e realiza operações CRUD;
- Program → Interface com o usuário.

Classe Aluno (Model)

- Classe que representa um aluno no sistema.
- Cada aluno tem um Id, Nome e Idade. Essa classe será usada em todas as operações CRUD.

```
namespace ProjetoConsoleBD.Models
{
    public class Aluno
    {
        public int Id { get; set; }
        public string Nome { get; set; }
        public int Idade { get; set; }
    }
}
```

Connection String

Definição da string de conexão para acessar o banco SQL Server.

- Integrated Security=True → usa autenticação do Windows
- User Id e Password → login SQL Server
- Encrypt=False;TrustServerCertificate=True → evita erros SSL

Exemplos:

- Windows Authentication:
Server=localhost\\SQLEXPRESS;Database=escola;Integrated Security=True;Encrypt=False;TrustServerCertificate=True;
- SQL Server Authentication: Server=192.168.0.39;Database=escola;User Id=aluno;Password=Aluno@123;Encrypt=False;TrustServerCertificate=True;

Inserir Aluno (Create)

- Método para adicionar um novo aluno ao banco.
- Usa SqlCommand com parâmetros para evitar SQL Injection. O método abre a conexão, executa o INSERT e fecha a conexão automaticamente com using.

```
public void Inserir(Aluno aluno)
{
    using (SqlConnection con = new SqlConnection(connectionString))
    {
        string query = "INSERT INTO Alunos (Nome, Idade) VALUES (@Nome, @Idade)";
        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@Nome", aluno.Nome);
        cmd.Parameters.AddWithValue("@Idade", aluno.Idade);

        con.Open();
        cmd.ExecuteNonQuery();
    }
}
```

Listar Alunos (Read)

- Método para buscar todos os alunos no banco.
- Usa SqlDataReader para percorrer os registros e transforma cada linha em um objeto Aluno.

```
public List<Aluno> Listar()
{
    List<Aluno> alunos = new List<Aluno>();

    using (SqlConnection con = new SqlConnection(connectionString))
    {
        string query = "SELECT Id, Nome, Idade FROM Alunos";
        SqlCommand cmd = new SqlCommand(query, con);

        con.Open();
        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            alunos.Add(new Aluno
            {
                Id = (int)reader["Id"],
                Nome = reader["Nome"].ToString(),
                Idade = (int)reader["Idade"]
            });
        }
    }

    return alunos;
}
```


Atualizar Aluno (Update)

- Método para atualizar os dados de um aluno existente.
- Recebe um objeto Aluno com o Id do aluno que será atualizado.
- Usa ExecuteNonQuery e verifica se algum registro foi afetado.

```
public void Atualizar(Aluno aluno)
{
    using (SqlConnection con = new SqlConnection(connectionString))
    {
        string query = "UPDATE Alunos SET Nome = @Nome, Idade = @Idade WHERE Id = @Id";
        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@Nome", aluno.Nome);
        cmd.Parameters.AddWithValue("@Idade", aluno.Idade);
        cmd.Parameters.AddWithValue("@Id", aluno.Id);

        con.Open();
        int linhasAfetadas = cmd.ExecuteNonQuery();
        if (linhasAfetadas == 0)
            Console.WriteLine("Nenhum aluno encontrado com o ID informado.");
    }
}
```

Deletar Aluno (Delete)

- Método para remover um aluno do banco.
- Recebe o Id do aluno e executa o DELETE. Se nenhum registro for afetado, exibe mensagem informando que o aluno não foi encontrado.

```
public void Deletar(int id)
{
    using (SqlConnection con = new SqlConnection(connectionString))
    {
        string query = "DELETE FROM Alunos WHERE Id = @Id";
        SqlCommand cmd = new SqlCommand(query, con);

        cmd.Parameters.AddWithValue("@Id", id);

        con.Open();
        int linhasAfetadas = cmd.ExecuteNonQuery();

        if (linhasAfetadas == 0)
            Console.WriteLine("Nenhum aluno encontrado com o ID informado.");
    }
}
```

Program.cs (Executando CRUD)

- Exemplo de como usar os métodos CRUD na prática. O usuário pode inserir, listar, atualizar e deletar alunos diretamente pelo console.

```
AlunoRepository repo = new AlunoRepository();

// Inserir aluno
Aluno novoAluno = new Aluno { Nome = "Maria", Idade = 20 };
repo.Inserir(novoAluno);

// Listar alunos
var alunos = repo.Listar();
foreach (var aluno in alunos)
    Console.WriteLine($"{aluno.Id} - {aluno.Nome} - {aluno.Idade}");

// Atualizar aluno
Aluno alunoAtualizado = new Aluno { Id = 1, Nome = "Maria Silva", Idade = 21 };
repo.Atualizar(alunoAtualizado);

// Deletar aluno
repo.Deletar(1);
```

Operações CRUD no SQL Server

- Resumo dos comandos SQL usados em cada operação.

Operação	Comando SQL
Create	INSERT INTO
Read	SELECT
Update	UPDATE
Delete	DELETE

Benefícios do CRUD com POO

Vantagens de organizar o código com Models, Repository e Program:

Código limpo, organizado e fácil de manter.

Separação clara entre dados, regras de acesso e interface.

Facilita inclusão de novas funcionalidades e testes.